

Machine Learning - 2026

Lecture – 1

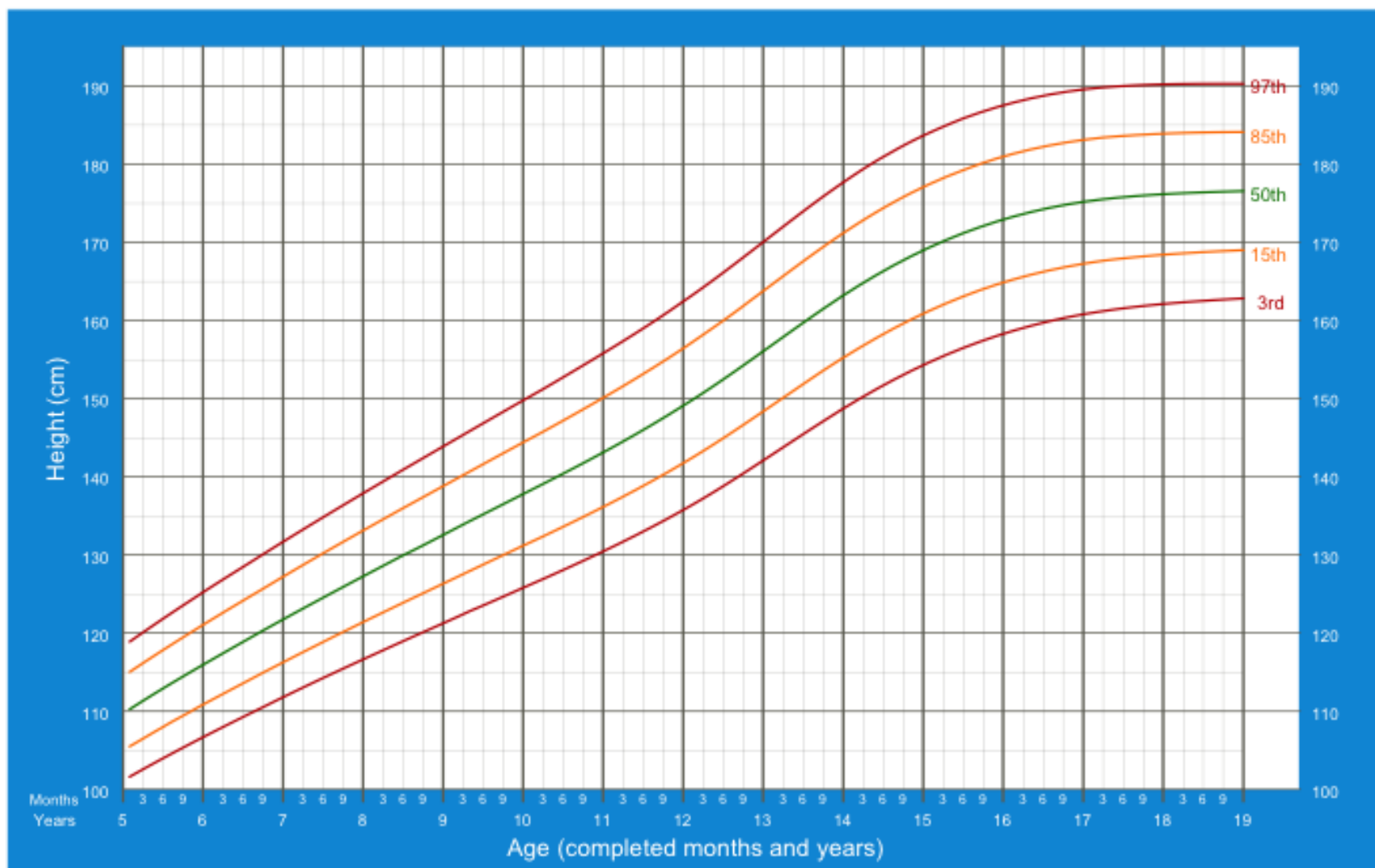
Linear Regression

Dr. Suresh Manandhar
Chair Professor of AI

Madan Bhandari University of Science and Technology

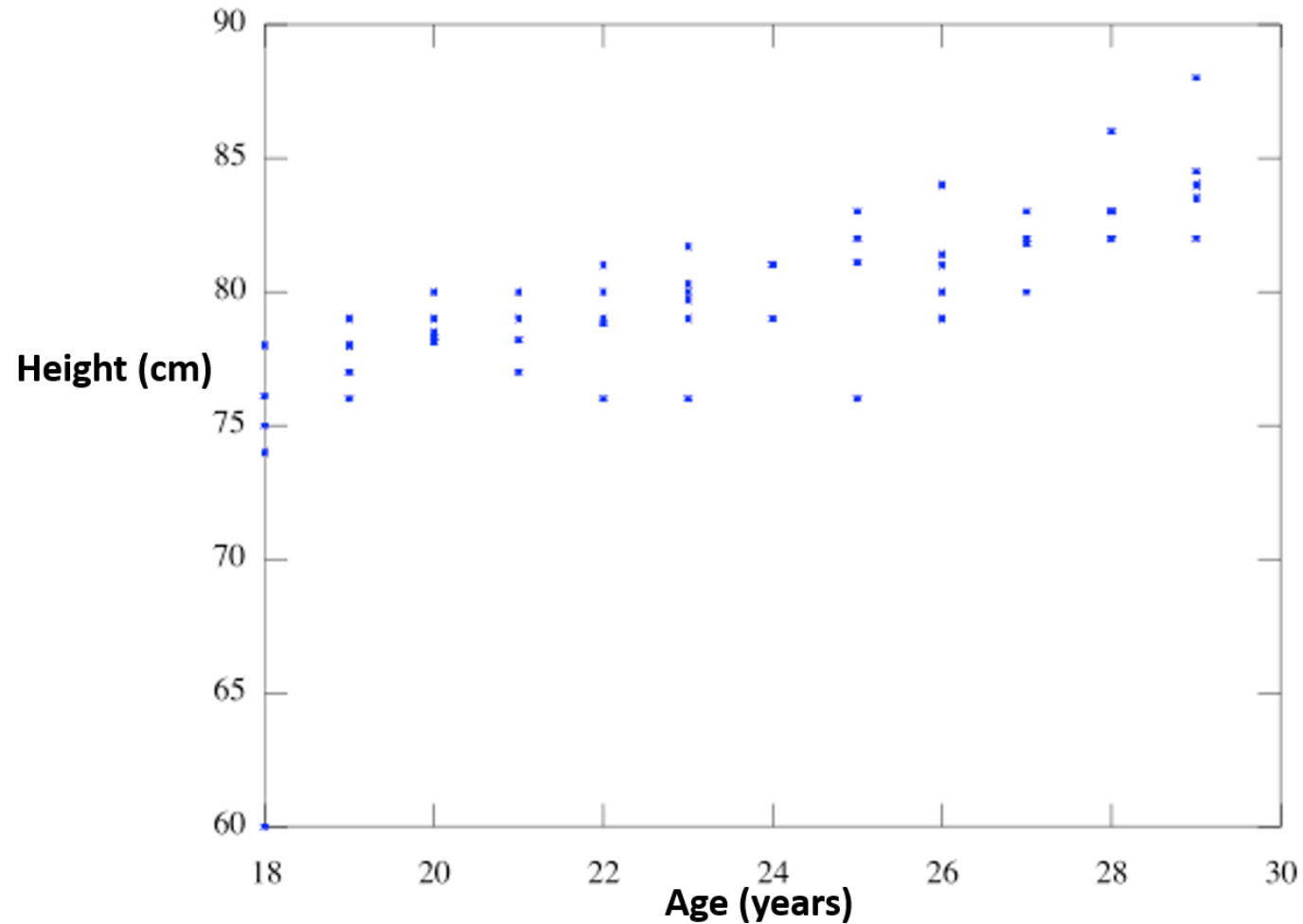
Height-for-age BOYS

5 to 19 years (percentiles)



2007 WHO Reference

Age vs. Height Scatter Plot



Linear vs Non-linear functions

- Linear functions refer to equations such as:
 - $f(\langle x, y \rangle) = 4x + 5y$
- Linear functions satisfy the properties of:
 - Additivity : $f(u + v) = f(u) + f(v)$
 - Homogeneity: $f(a \cdot v) = a \cdot f(v)$
- Non-linear functions refer to equations such as:
 - $f(\langle x, y \rangle) = 2x^2 + 3y$
- Non-linear equations (in general) do not satisfy the properties of additivity and homogeneity.
- If we assume x^2 and y are known and are fixed:
 - $f(\langle a, b \rangle) = ax^2 + by$ is linear
- Thus, in general, polynomials will be linear w.r.t their co - efficient but non-linear w.r.t to their variables.

Linear Regression

- First approximation:
 - Aim to fit polynomial equations to given data
 - e.g. $\text{Height} = 100 + 0.2 \text{ Age} - 0.01 \text{ Age}^2$
 $\text{Height} = 100 + 0.2 \text{ Age} - 0.01 \text{ Age}^2 - 0.02 \text{ Weight}^2$
- Example:
 - Degree-2 polynomial in 1 'variable' can be written as:
 $f(a_0, a_1, a_2) = a_0 + a_1x + a_2x^2$
 - $f(a_0, a_1, a_2)$ is **linear** since it satisfies the properties of additivity and homogeneity

Linear Regression

- In linear regression, the function to be used is fixed in advance.
- Thus, we decide in advance whether to use a 2-degree or 3-degree polynomial.
- This is a manual task – although this is not much of a restriction
- For example, we can decide to use a 100th- degree polynomial.
- Remember, a 100th degree polynomial includes all lower degree polynomials

Linear Regression – better definition

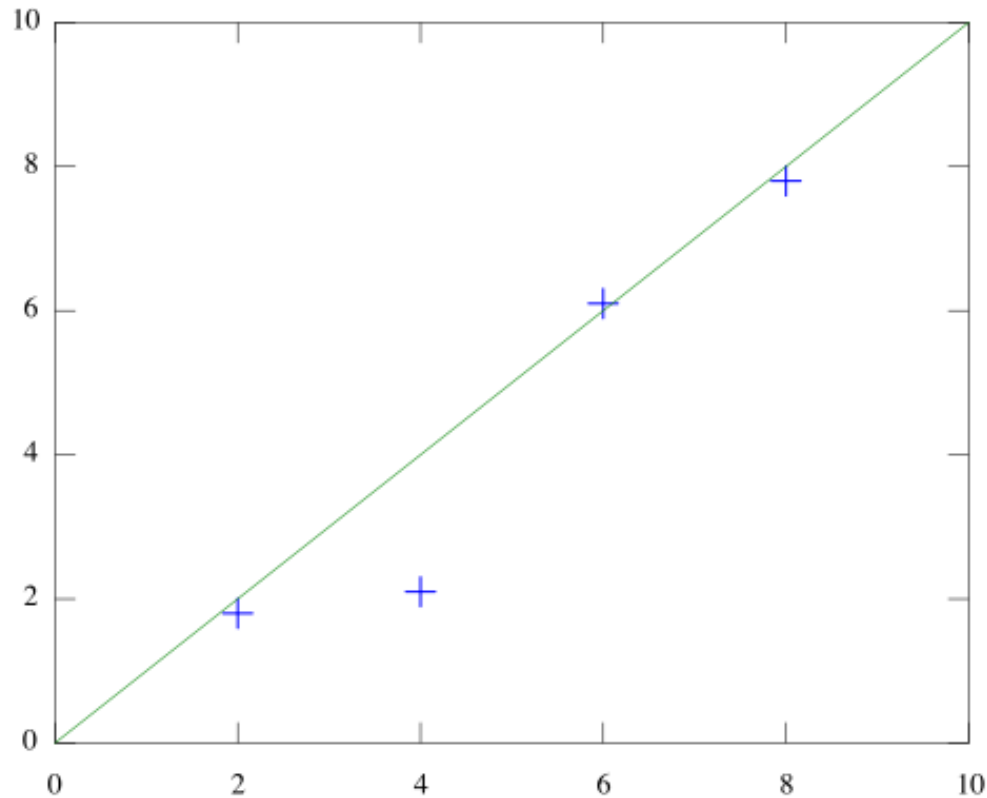
Aim to:

1. find '**linear co - efficient**' of a known function
2. that fits the given data as '**best**' as possible

We need to define what 'best' means.

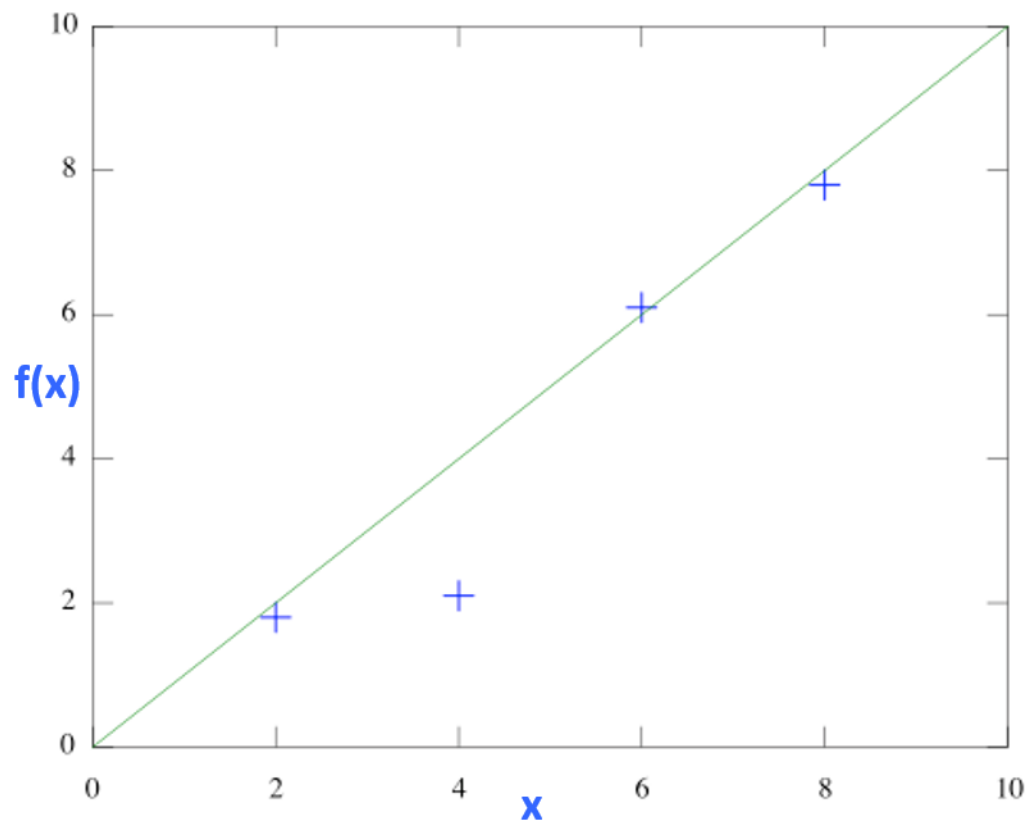
Error

- One way to measure the 'fit' of the equation with respect to the given data will be to measure the error.



x	y
2	1.8
4	2.1
6	6.1
8	7.8

Calculating Error



x	y	$f(x)=x$	$\text{Err} = y - f(x)$
2	1.8	2	-0.2
4	2.1	4	-1.9
6	6.1	6	0.1
8	7.8	8	-0.2

- Simply adding the errors will **not** work.
- **Why?**

Loss functions for regression

- *Absolute value loss:*
 - $|y - f(x)|$
- *Squared loss:*
 - $(y - f(x))^2$
- *ε -sensitive loss:*
 - $\max(|y - f(x)| - \varepsilon, 0)$
[only consider error $> \varepsilon$]

x	y	f(x)=x	Err = y-f(x)
2	1.8	2	-0.2
4	2.1	4	-1.9
6	6.1	6	0.1
8	7.8	8	-0.2

Total loss is given by :

$$\sum_i \text{loss}(y_i, f(x_i))$$

Least squares regression

- Linear regression with ‘squared loss’ function is also known as ‘least squares regression’
- Most studied regression method
- Has some nice mathematical and computational properties
(– we will look at some of these later)

Vector notation

- Let $f(\mathbf{X}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots$
- Then $f(\mathbf{X})$ can be written in vector form as:
 - $f(\mathbf{X}) = \theta \cdot \mathbf{X}$
 - where $\mathbf{X} = \langle 1, x_1, x_2, \dots \rangle$ and
 - $\theta = \langle \theta_0, \theta_1, \theta_2, \dots \rangle$
 - $\theta \cdot \mathbf{X}$ denotes the '**dot product**' or the 'inner product' between vectors θ and $\mathbf{X}$
- For example, given the following function:
 - $-20 + 10x + 3x^2 + 12w$ can be written as $\mathbf{X} \cdot \theta$:
 - $\mathbf{X} = \langle 1, x, x^2, w \rangle$ and $\theta = \langle -20, 10, 3, 12 \rangle$
 - e.g. assume that $x = \text{age}$ and $w = \text{weight}$ and we are trying to predict height

Vector notation - contd

- **Constant term:** polynomials such as

$-20 + 10x + 3x^2 + 12w$ contain the constant term

$$\theta_0 = -20.$$

where, $X = \langle 1, x, x^2, w \rangle$ and $\theta = \langle -20, 10, 3, 12 \rangle$

The corresponding data term $x_0 = 1$ (always)

- For this example, the input data X will be just x (say age) and w (say weight) which we convert to
 $X = \langle 1, x, x^2, w \rangle$
- Input data can then be written as:
 - $D = \{ (X_1, y_1), (X_2, y_2), (X_3, y_3), \dots \}$
- For a given θ , total Loss can be calculated:

$$L(\theta) = \sum_i \text{loss}(y_i, \theta \cdot X_i)$$

Least squares regression

- Solving the regression problem amounts to finding the minimizer for the total loss
- Let θ^* denote the actual solution we wish to find
- Then:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} \sum_i \operatorname{loss}(y_i, \theta \cdot X_i)$$

- We can view as a function of θ then seek to find the **minima** of this function

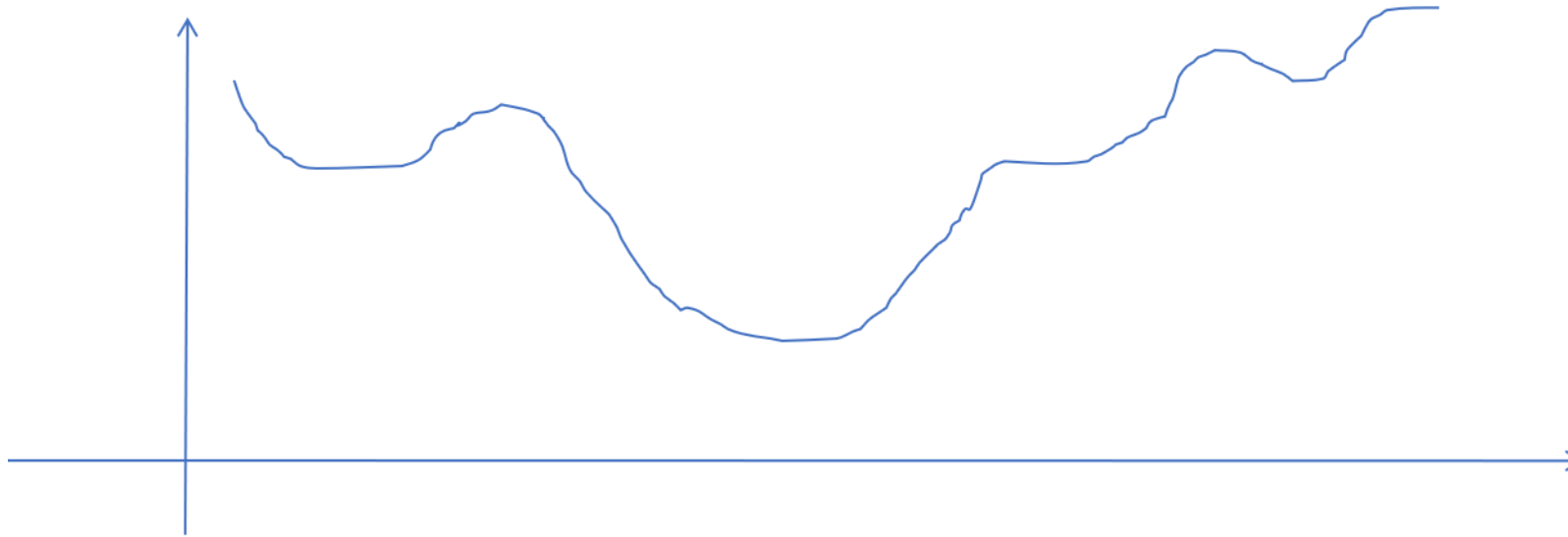
$$\sum_i \operatorname{loss}(y_i, \theta \cdot X_i)$$

- i.e. find **minima** of

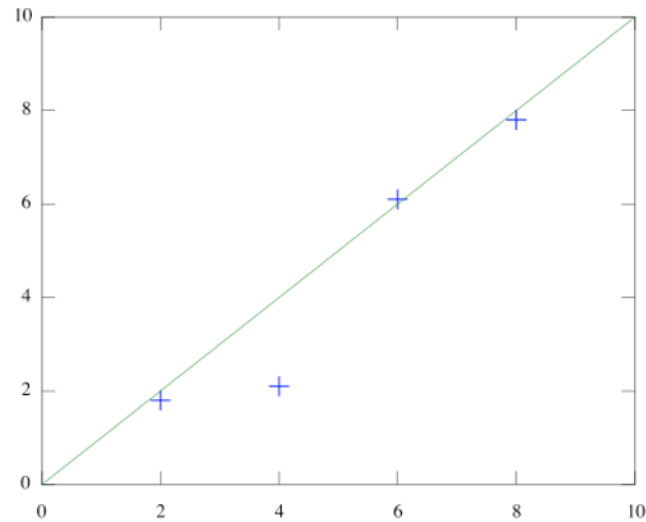
$$L(\theta) = \sum_i \operatorname{loss}(y_i, \theta \cdot X_i)$$

Shape of $L(\theta)$

- Does $L(\theta)$ have a single global minimum?
- Can we get some idea of what $L(\theta)$ might look like?
- What if $L(\theta)$ has local minima?



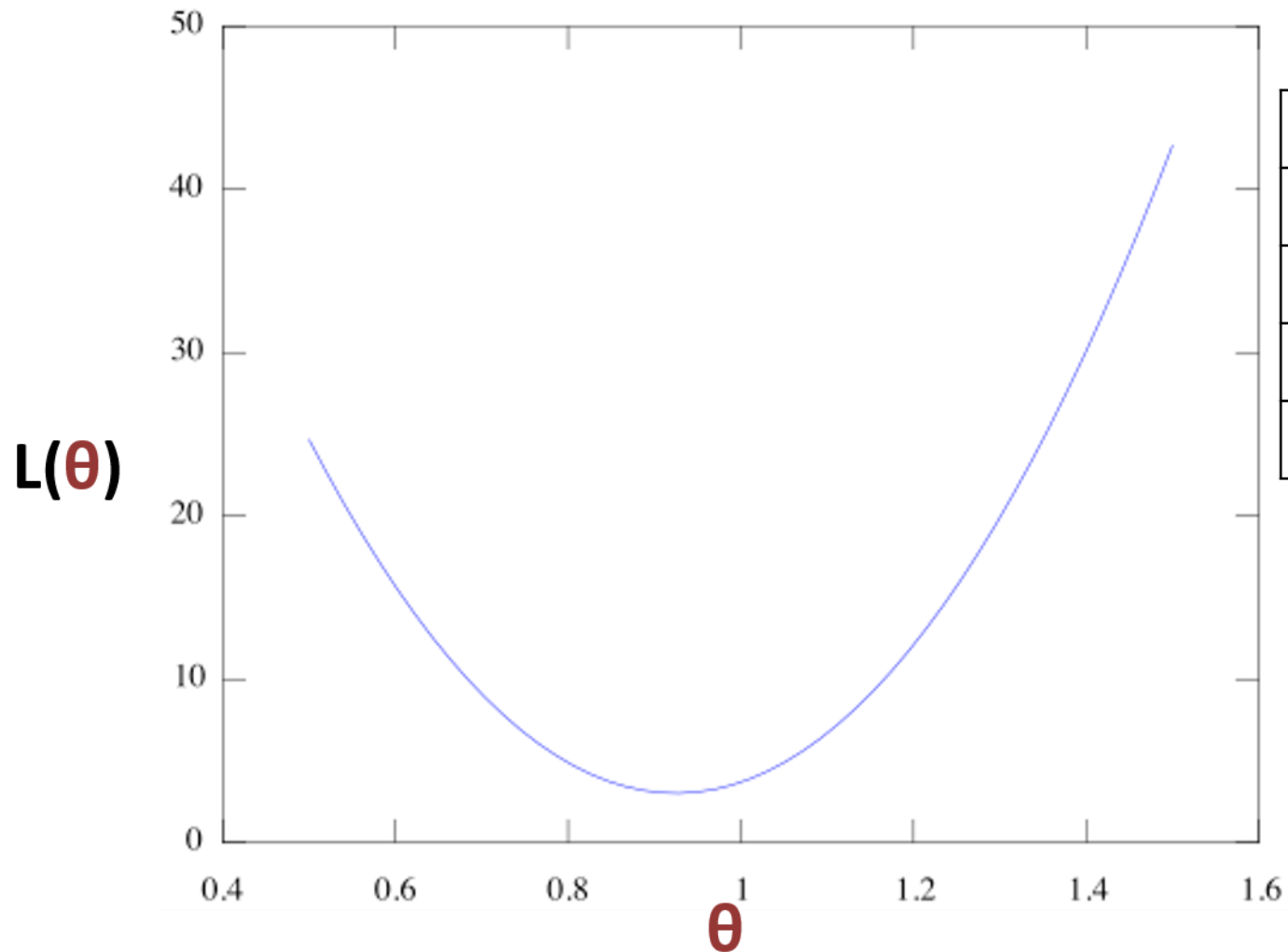
Example



$$f(x) = \theta \cdot x$$

x	y	Loss ($\theta=1$)	Loss ($\theta=1.1$)	Loss ($\theta=0.9$)
2	1.8	0.04	0.16	0
4	2.1	3.61	5.29	2.25
6	6.1	0.01	0.25	0.49
8	7.8	0.04	1	0.36
	Total	3.7	6.7	3.1

Example



x	y
2	1.8
4	2.1
6	6.1
8	7.8

$$f(x) = \theta x$$

$$L(\theta) = \sum_i \text{loss}(y_i, \theta \cdot x_i)$$

Convexity

- Functions such as $L(\theta)$ in example earlier are called **convex**
- Convex functions have the characteristic bell shape
- Guaranteed to have a unique **global minimum**
- Not a rigorous proof of convexity yet!

Finding the minima of $L(\theta)$

- Many techniques exist
- Simplest and easiest to implement is **gradient descent**
- Gradient descent/ascent simply attempts to follow the curve in the **direction of descent**

Gradient Descent

- Start at any point in the curve
 - Repeat:
 - Calculate gradient
 - Take a step in the direction of descent
 - Reach a new position of the curve
- Until stopping criteria is reached

Algorithm

- Initialize θ with zeros or random values
- Repeat:
 - Calculate $L(\theta)$
 - Calculate gradient $\Delta L(\theta)$ at current θ
 - Update: $\theta := \theta - \text{Learning Rate} * \Delta L(\theta)$ until termination criteria
- Output θ

Calculating gradient $\Delta L(\theta)$

- We can employ differential calculus to compute the gradient of $L(\theta)$ in closed form

$$\begin{aligned} L(\theta) &= \sum_i \text{loss}(y_i, \theta \cdot X_i) \\ &= \sum_i (y_i - \theta \cdot X_i)^2 \end{aligned}$$

$$\frac{\delta L(\theta)}{\delta \theta_j} = ?? \quad (\text{Class work})$$

Calculating gradient $\Delta L(\theta)$

- We can employ differential calculus to compute the gradient of $L(\theta)$ in closed form

$$\begin{aligned} L(\theta) &= \sum_i \text{loss}(y_i, \theta \cdot X_i) \\ &= \sum_i (y_i - \theta \cdot X_i)^2 \end{aligned}$$

$$\begin{aligned} \frac{\delta L(\theta)}{\delta \theta_j} &= \frac{\delta}{\delta \theta_j} \sum_i (y_i - \theta \cdot X_i)^2 \\ &= \sum_i -2(y_i - \theta \cdot X_i) \frac{\delta (\theta_0 + \theta_1 x_{i1} + \theta_2 x_{i2} + \dots)}{\delta \theta_j} \\ &= \sum_i -2(y_i - \theta \cdot X_i) x_{ij} \end{aligned}$$

Calculating gradient $\Delta L(\theta)$

- This gives us the gradient of $L(\theta)$ along each θ_j
- The gradient vector :

$$\Delta L(\theta) = \left\langle \frac{\delta L(\theta)}{\delta \theta_1}, \frac{\delta L(\theta)}{\delta \theta_2}, \frac{\delta L(\theta)}{\delta \theta_3}, \dots \right\rangle$$

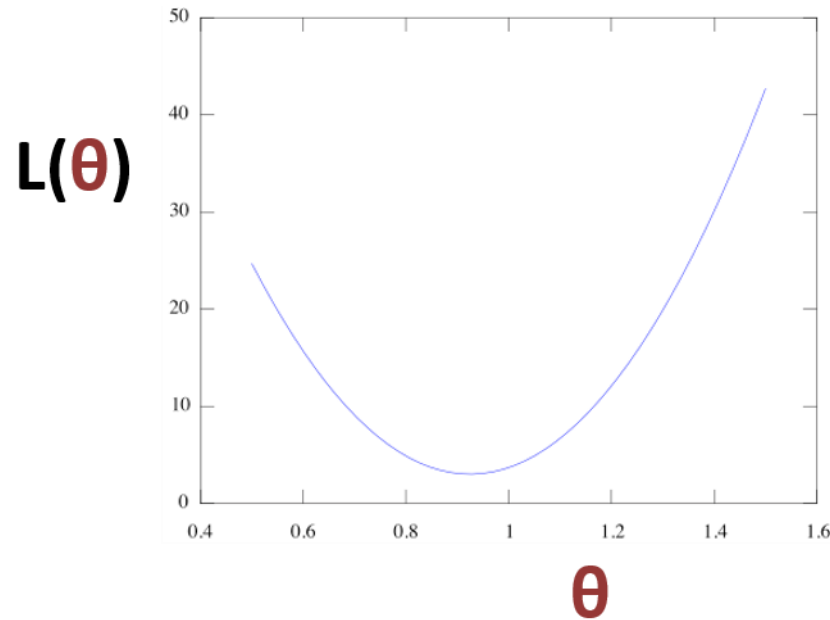
- This can be computed more efficiently, in a vectorized implementation:

$$\Delta L(\theta) = \sum_i -2(y_i - \theta \cdot X_i) \cdot X_i \quad (\text{note -ve sign})$$

Algorithm (more details)

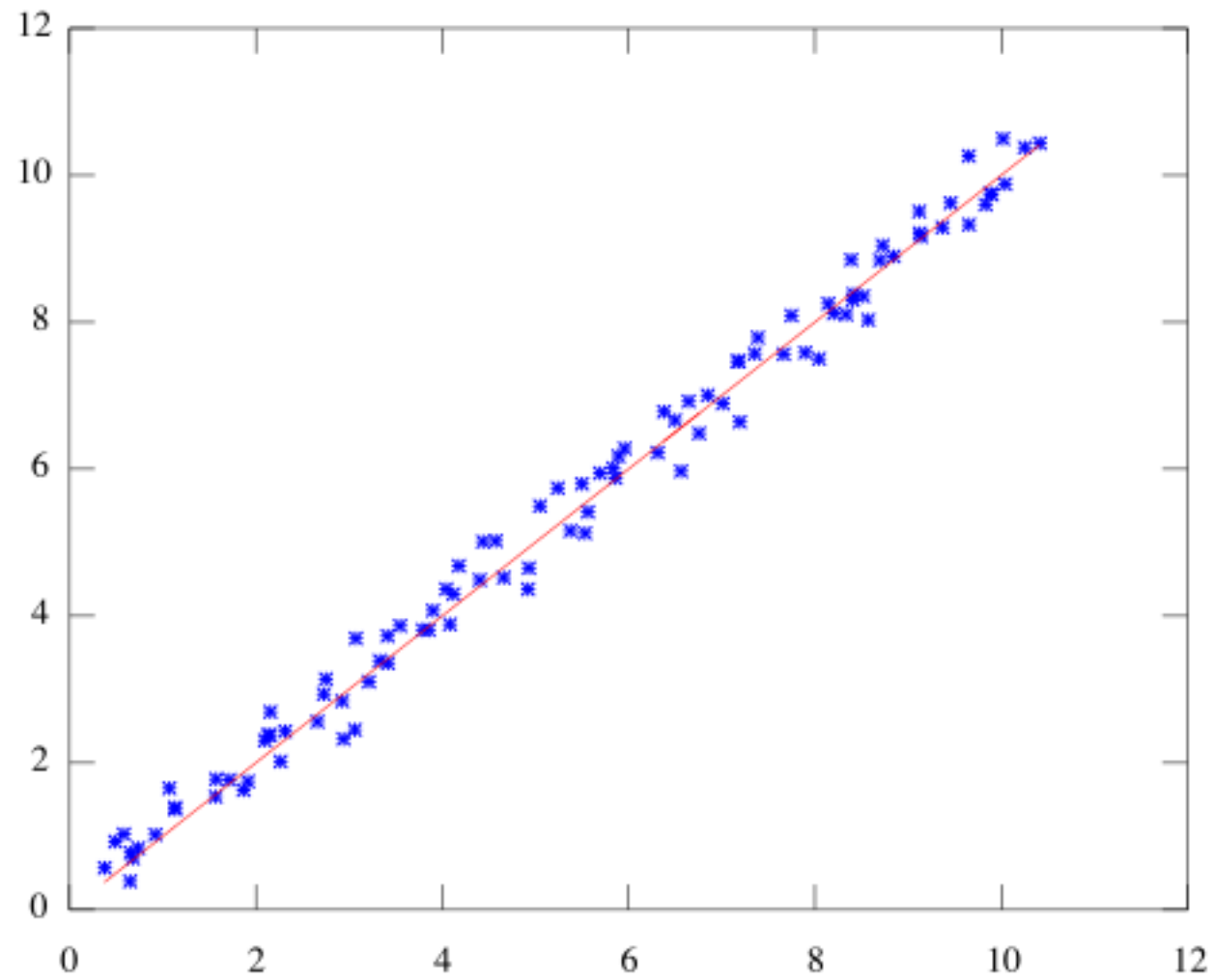
- Initialize θ with zeros or random values
- Read X, Y (input data)
- Repeat:
 - Initialize $L(\theta) = 0, \Delta L(\theta) = 0$
 - For $\langle X_i, y_i \rangle$:
 - $L(\theta) += (y_i - \theta \cdot X_i)^2$
 - $\Delta L(\theta) += (y_i - \theta \cdot X_i) \cdot X_i$ (removed -ve sign)
 - Update: $\theta := \theta + \text{Learning Rate} * \Delta L(\theta)$ (changed - to +)
- Output θ

Learning Rate

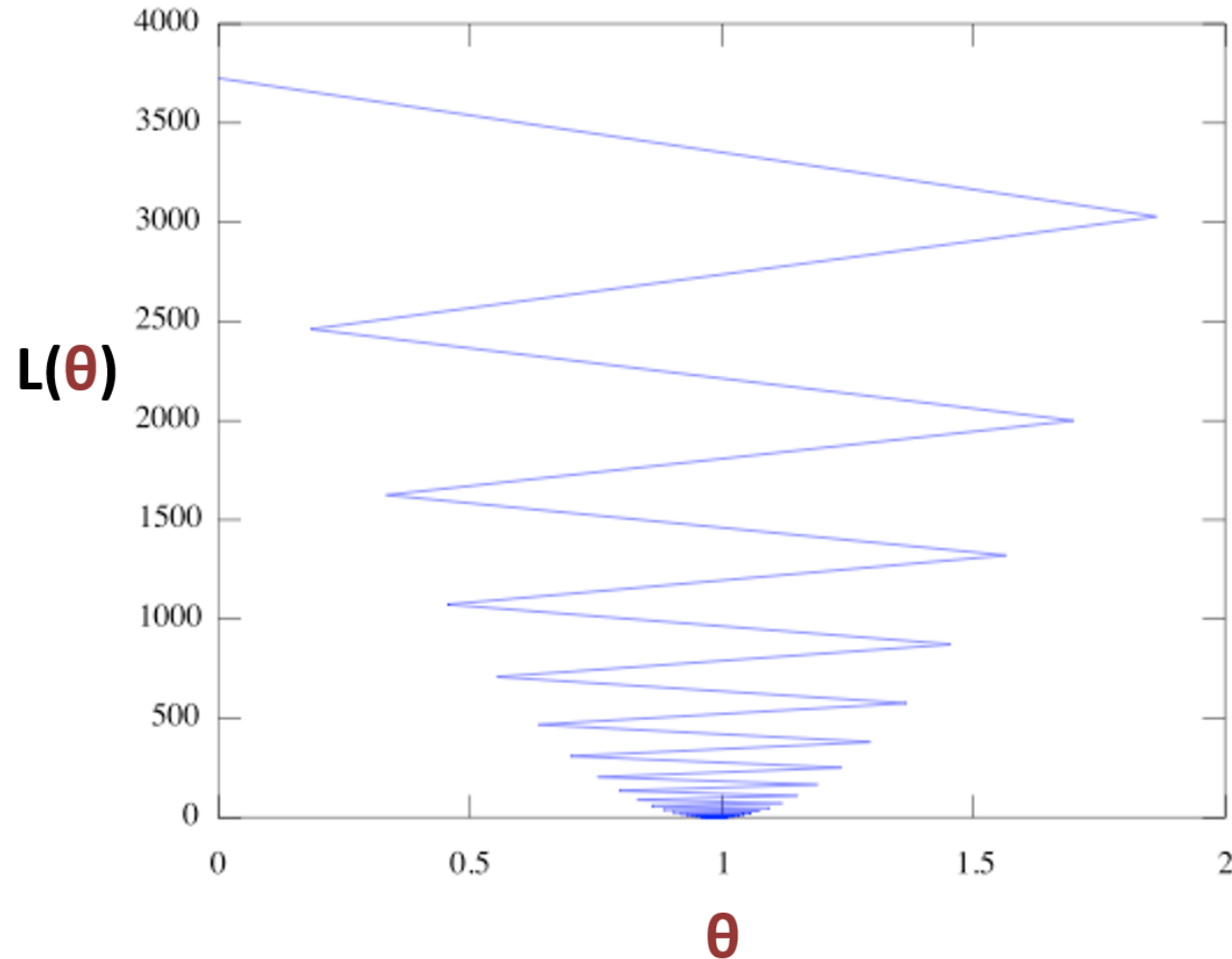


Update rule: $\theta := \theta - \text{Learning Rate} * \Delta L(\theta)$

- Choosing large values of Learning Rate
 - bigger jumps, faster, but may miss the minima
- Choosing small values of Learning Rate
 - smaller moves, slower, less likely to miss the minima



Gradient descent in action

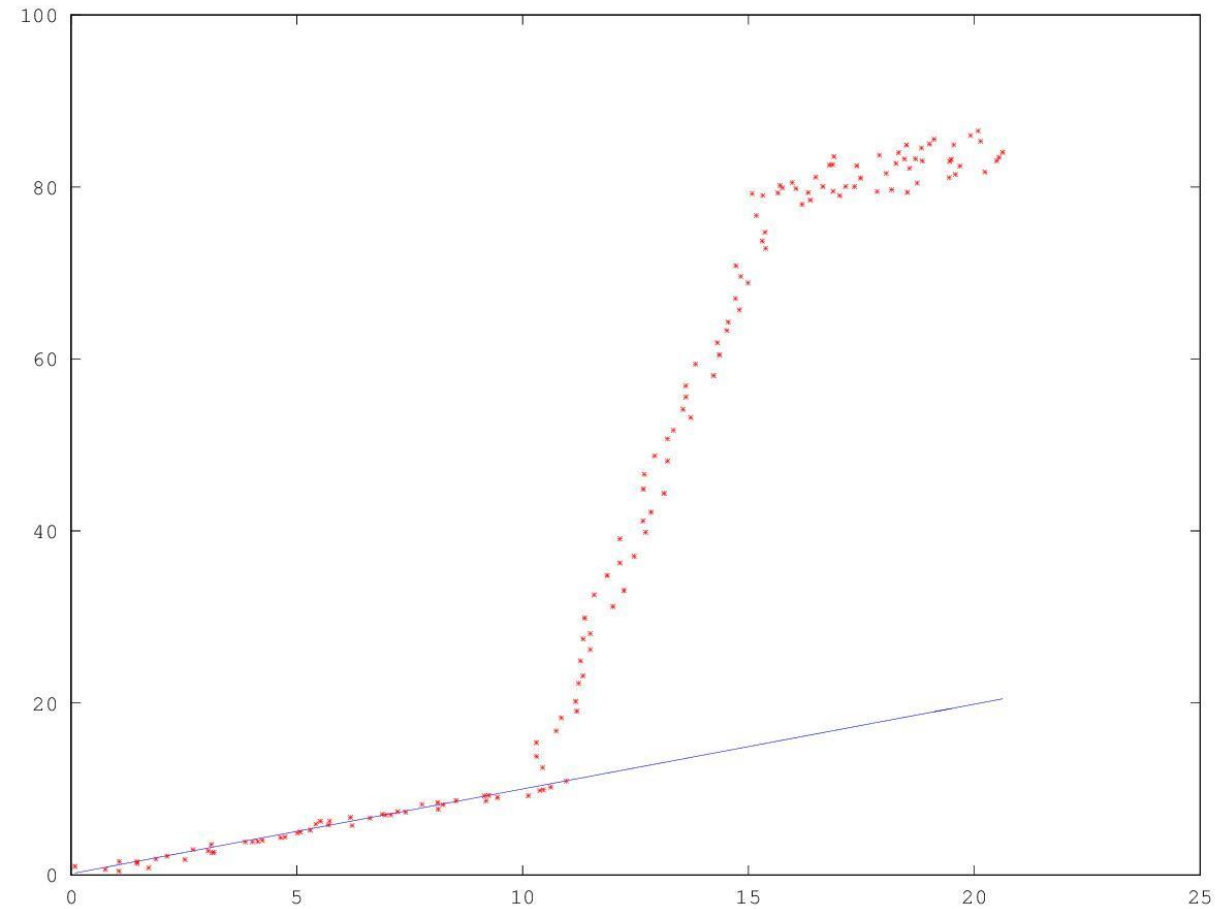


Issues with gradient descent

- Need to guess Learning Rate
- Need to guess Stopping Criteria/Does not detect convergence
- Can be very slow
- Better methods exists such as BGFS and L-BGFS that are built into existing optimization software (MatLab, Octave, Python/scipy)
- These does not require pre-specifying learning rate or stopping criteria.
- Practical!

Issues with simple gradient descent

- Fails to find global optimal when gradient is shallow
- Unable to alter step size dynamically
- Higher precision floating point arithmetic needed



Exact Solution in Closed Form

- Use matrix formulation to derive solution in closed form
- Represent input data as $\mathbf{X}$ consisting of $\mathbf{N}$ data points each with $\mathbf{K}$ dimensions
- Thus, input data $\mathbf{X}$ is $\mathbf{K} \times \mathbf{N}$ dim matrix.
- Represent $\boldsymbol{\theta}$ by $\mathbf{K}$ -dim vector i.e. $\mathbf{K} \times 1$ dim matrix
- Thus $\boldsymbol{\theta} = \langle \theta_0, \theta_1, \theta_2, \dots \rangle$ (vector notation)
 - $\boldsymbol{\theta}^T = [\theta_0, \theta_1, \theta_2, \dots]$ (matrix notation)
- Dot product $\mathbf{X} \cdot \boldsymbol{\theta}$ becomes $\mathbf{X}\boldsymbol{\theta}$ in matrix notation (**order important**)

Solution in Closed Form

$$\begin{aligned}\Delta L(\theta) &= \sum_i -2(y_i - \theta \cdot X_i) \cdot X_i \\ &= -2 \sum_i X_i \cdot (y_i - X_i \cdot \theta)\end{aligned}$$

X is a $N \times K$ matrix (N data points)

Y is a $N \times 1$ matrix (has just one column)

Both $\Delta L(\theta)$, θ are $K \times 1$ matrix (is a K -dim vector)

$X\theta$ is a $N \times 1$ matrix (has just one Column)

X^T is a $K \times N$ matrix

$X^T X$ is a $K \times K$ matrix

$$\Delta L(\theta) = -2 X^T (Y - X\theta) \quad (\text{is a } K \times 1 \text{ dim matrix})$$

Details

- Error matrix ($\mathbf{N} \times 1$) can be written as:

$$\text{Error}_i = y_i - X_i \cdot \theta \quad i$$

or, $\text{Error} = Y - X\theta$

$$\begin{aligned} \Delta L(\theta) &= -2 \sum_i X_i \cdot (y_i - X_i \cdot \theta) \\ &= -2 \sum_i X_i \cdot \text{Error}_i \\ &= -2 X^T \text{Error} \\ &= -2 X^T (Y - X\theta) \end{aligned}$$

- if X^T is $1 \times N$ and Y is a $N \times 1$ then $X^T Y = \sum_i X_i \cdot Y_i$
- this generalises to the case when X^T is $K \times N$ then $X^T Y$ is $K \times 1$
- similarly, $X\theta$ is a $N \times 1$ matrix so has same shape as Y
- X^T is $K \times N$ dim. and $(Y - X\theta)$ is $N \times 1$ dim. hence $X^T (Y - X\theta)$ is $K \times 1$

We have, $\Delta L(\theta) = -2 (X^T Y - X^T X \theta)$

Setting $\Delta L(\theta) = 0$ (since, derivative is 0 at extremum points)

$$-2 (X^T Y - X^T X \theta) = 0$$

$$X^T Y = X^T X \theta$$

$$(X^T X)^{-1} X^T Y = (X^T X)^{-1} X^T X \theta$$

$$(X^T X)^{-1} X^T Y = \theta$$

Hence: $\theta = (X^T X)^{-1} X^T Y$

gives *closed form* solution

But requires $X^T X$ to be *invertible*

Issues with Closed form solution

- For matrices inversion would be too slow
- For large datasets this may not be a practical solution
- Not all matrices are invertible – (although not an issue with regularization : coming later)
- Hence, gradient based methods such as variations on BFGS (L-BFGS) are popular choice

Learning outcomes summary

- Understand the purpose/goal of linear regression
- Why it is called 'linear'
- The different ways to measure error
- The least squares loss function
- Why this loss function is convex (at least intuitively)
- Derive gradient for the loss function
- Implement gradient-based algorithm
- Issues with simple gradient based formulation
- Derivation of closed-form solution.
- Issues with closed-form solution.

Thank You